

✓ Lista de Exercícios — Introdução ao LangChain

Disciplina: Generative AI & Advanced Analytics

Instituição: PUC Minas

Professor: Renan Santos Mendes

Email: renansantosmendes@gmail.com

Aluno(a): Matheus Oliveira Nonato da Silva

Matrícula: 241259

Instruções

- Esta lista possui **10 questões práticas** sobre o uso do LangChain, com base no conteúdo visto em aula (Messages, Templates, Runnable, Chains e Structured Output).
- Sempre que encontrar `...` (reticências), é porque **você** precisa escrever o código.
- Rode as células em ordem — algumas questões dependem de variáveis criadas em questões anteriores (`llm`, `chain`, etc.).
- Não é necessário fornecer uma chave de API válida da OpenAI: usaremos o **proxy da disciplina**, autenticando com a sua matrícula.
- Ao final, salve o notebook e envie conforme instruções do professor.

✓ Configuração do ambiente

Vamos instalar as bibliotecas necessárias: `langchain-openai` (integração do LangChain com modelos compatíveis com a API da OpenAI) e `pgl-auth` (cliente de autenticação da disciplina, usado para gerar o token de acesso ao proxy).

```
!uv pip install langchain-openai pgl-auth -q
```

✓ Autenticação e criação do modelo de linguagem

Utilizamos o `PGLAuthClient` para autenticar com a matrícula e senha (armazenadas nos *secrets* do Colab) e obter um token JWT. Esse token é usado como `api_key` do `ChatOpenAI`, que aponta para o proxy da disciplina (`base_url`).

Essas células já estão prontas — é o mesmo padrão utilizado nas aulas práticas. Basta executá-las.

```
from google.colab import userdata
from pgl_auth import PGLAuthClient

token = PGLAuthClient().login(
    registration_number=userdata.get("PGL_REGISTRATION_NUMBER"),
    password=userdata.get("PGL_PASSWORD"),
)
```

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    base_url="https://pgl-proxy.vercel.app/v1",
    api_key=token,
    model="gpt-4o-mini",
    temperature=0.2,
)

# Teste rápido para confirmar que o modelo está respondendo
print(llm.invoke("Diga oi em uma frase curta.").content)

Oi! Como você está?
```

✓ Questão 1 — Primeiro contato com o `invoke`

Use o objeto `llm` já criado para enviar, como texto simples, a pergunta:

"O que é uma variável em programação? Responda em no máximo duas frases."

Armazene o resultado na variável `response_q1` e imprima apenas o conteúdo textual da resposta (o atributo `.content`).

```
response_q1 = llm.invoke("O que é uma variável em programação? Responda em no máximo duas frases.")  
print(response_q1.content)
```

Uma variável em programação é um espaço na memória que armazena dados e pode ser referenciado por um nome. Ela permite que

❯ Questão 2 — Trabalhando com `SystemMessage`, `HumanMessage` e `AIMessage`

Monte uma lista de mensagens chamada `messages_q2` contendo:

1. Um `SystemMessage` instruindo o modelo a atuar como um **tutor de programação que sempre responde com exemplos de código em Python**.
2. Um `HumanMessage` perguntando: "Como faço um laço de repetição que imprime os números de 1 a 5?"

Em seguida, invoque o `llm` com essa lista e imprima o conteúdo da resposta.

Dica: importe as classes de `langchain_core.messages`.

```
from langchain_core.messages import AIMessage, HumanMessage, SystemMessage  
  
messages_q2 = [  
    SystemMessage(content="Você é um tutor de programação que sempre responde com exemplos de código em Python."),  
    HumanMessage(content="Como faço um laço de repetição que imprime os números de 1 a 5?")  
]  
  
response_q2 = llm.invoke(messages_q2)  
print(response_q2.content)
```

Você pode usar um laço de repetição ``for`` ou ``while`` para imprimir os números de 1 a 5 em Python. Aqui estão exemplos de am

Usando um laço ``for``:

```
```python  
for i in range(1, 6):
 print(i)
```
```

Usando um laço ``while``:

```
```python  
i = 1
while i <= 5:
 print(i)
 i += 1
```
```

Ambos os exemplos irão imprimir os números de 1 a 5. Você pode escolher o que achar mais conveniente!

❯ Questão 3 — Simulando um histórico de conversa

Reaproveite a lista `messages_q2` da questão anterior e monte uma nova lista chamada `conversation_history_q3` adicionando, na sequência:

3. Um `AIMessage` simulando que o modelo já respondeu: "Você pode usar um `for` combinado com a função `range`."
4. Um novo `HumanMessage` perguntando: "E como eu faria isso com um `while`?"

Invoque o `llm` com `conversation_history_q3` e imprima a resposta. Observe como o modelo utiliza o contexto da conversa anterior para responder de forma coerente.

```
conversation_history_q3 = [  
    SystemMessage(content="Você é um tutor de programação que sempre responde com exemplos de código em Python."),  
    HumanMessage(content="Como faço um laço de repetição que imprime os números de 1 a 5?"),  
    SystemMessage(content="Você pode usar um for combinado com a função range."),  
    HumanMessage(content="E como eu faria isso com um while?"),  
]
```

```
]

response_q3 = llm.invoke(conversation_history_q3)
print(response_q3.content)
```

Você pode usar um laço `while` da seguinte forma:

```
```python
numero = 1

while numero <= 5:
 print(numero)
 numero += 1
```
```

Neste código, começamos com `numero` igual a 1 e continuamos o laço enquanto `numero` for menor ou igual a 5. A cada iteraç

▼ Questão 4 — PromptTemplate

Crie um `PromptTemplate` chamado `prompt_template_q4` a partir do seguinte texto, com uma variável `{topico}`:

"Explique o conceito de {topico} em programação, usando uma linguagem simples, como se estivesse explicando para um iniciante."

Formate o template para `topico="recursão"` e imprima o texto resultante. Em seguida, envie esse texto formatado diretamente para o `llm` e imprima o conteúdo da resposta.

```
from langchain_core.prompts import PromptTemplate

prompt_template_q4 = PromptTemplate(
    template="Explique o conceito de {topico} em programação, usando uma linguagem simples, como se estivesse explicando p
    input_variables=["topico"],
)

formatted_prompt_q4 = prompt_template_q4.format(topico="recursão")
print(formatted_prompt_q4)

response_q4 = llm.invoke(formatted_prompt_q4)
print(response_q4.content)
```

Explique o conceito de recursão em programação, usando uma linguagem simples, como se estivesse explicando para um iniciant
Claro! Vamos imaginar que você está tentando resolver um problema que pode ser dividido em partes menores, e essas partes m

****O que é recursão?***

Recursão é uma técnica onde uma função se chama a si mesma para resolver um problema. É como se você estivesse tentando res

****Como funciona?***

1. ****Caso Base****: Primeiro, você precisa de um caso base, que é a condição que diz quando a função deve parar de se chamar.

2. ****Chamada Recursiva****: Se o problema não estiver resolvido (ou seja, você não chegou ao caso base), a função se chama no

****Exemplo Prático****

Vamos usar um exemplo simples: calcular o fatorial de um número. O fatorial de um número (n!) é o produto de todos os númer

- Fatorial de 5 (5!) = 5 × 4 × 3 × 2 × 1

Usando recursão, podemos definir a função assim:

1. ****Caso Base****: Se n é 1, o fatorial de 1 é 1.
2. ****Chamada Recursiva****: Se n é maior que 1, o fatorial de n é n multiplicado pelo fatorial de (n-1).

Aqui está como isso ficaria em código:

```
```python
def fatorial(n):
 if n == 1: # Caso base
 return 1
 else:
 return n * fatorial(n - 1) # Chamada recursiva
```
```

Se você chamar `fatorial(5)`, a função vai se chamar assim:

```
- fatorial(5) → 5 * fatorial(4)
- fatorial(4) → 4 * fatorial(3)
- fatorial(3) → 3 * fatorial(2)
- fatorial(2) → 2 * fatorial(1)
- fatorial(1) → 1 (a função para aqui)
```

Depois, a função vai voltar e calcular tudo:

```
- fatorial(2) retorna 2
- fatorial(3) retorna 6
- fatorial(4) retorna 24
- fatorial(5) retorna 120
```

E assim, você obtém o resultado final!

****Resumindo**:** Recursão é uma maneira de resolver problemas chamando a função que você está escrevendo dentro dela mesma, a

❏ Questão 5 — ChatPromptTemplate

Crie um `ChatPromptTemplate` chamado `chat_prompt_template_q5` com duas mensagens:

- Uma mensagem de sistema: "Você é um revisor de código Python, direto e objetivo."
- Uma mensagem humana com uma variável `{codigo}`: "Revise o seguinte trecho de código e aponte possíveis problemas:\n\n{codigo}"

Formate as mensagens (`format_messages`) usando o seguinte trecho de código como valor de `codigo`:

```
def soma(a, b)
    return a+b
```

Imprima cada mensagem formatada e depois invoque o `llm` com o resultado, imprimindo a resposta.

```
from langchain_core.prompts import ChatPromptTemplate

chat_prompt_template_q5 = ChatPromptTemplate.from_messages(
    [
        ("system", "Você é um revisor de código Python, direto e objetivo."),
        ("human", "Revise o seguinte trecho de código e aponte possíveis problemas:\n\n{codigo}")
    ]
)

codigo_exemplo = '''def soma(a, b)
    return a+b'''

formatted_messages_q5 = chat_prompt_template_q5.format_messages(codigo=codigo_exemplo)
for message in formatted_messages_q5:
    print(message)

response_q5 = llm.invoke(formatted_messages_q5)
print(response_q5.content)
```

```
content='Você é um revisor de código Python, direto e objetivo.' additional_kwargs={} response_metadata={}
content='Revise o seguinte trecho de código e aponte possíveis problemas:\n\ndef soma(a, b)\n    return a+b' additional_kwa
0 trecho de código apresenta alguns problemas. Aqui estão as correções necessárias:
```

1. ****Falta de dois pontos**:** A definição da função `soma` deve terminar com dois pontos (`:`).
2. ****Indentação**:** O corpo da função deve estar corretamente indentado, embora neste caso específico não haja problema, é u

Aqui está a versão corrigida do código:

```
```python
def soma(a, b):
 return a + b
```
```

Além disso, considere adicionar verificações de tipo para garantir que `a` e `b` sejam números, caso isso seja relevante pa

❏ Questão 6 — Entendendo Runnable

Todo componente do LangChain que pode ser executado implementa a interface `Runnable`, expondo métodos como `invoke`, `batch` e `stream`.

a) Verifique, usando `hasattr`, se `chat_prompt_template_q5` e `llm` possuem os métodos `invoke`, `batch` e `stream`. Imprima os resultados de forma organizada.

b) Use o método `batch` do `llm` para enviar, de uma só vez, a lista de perguntas abaixo, armazenando o resultado em `batch_responses_q6`. Em seguida, imprima o conteúdo de cada resposta.

```
perguntas_q6 = [  
    "O que é uma lista em Python?",  
    "O que é uma tupla em Python?",  
    "O que é um dicionário em Python?",  
]
```

```
for metodo in ["invoke", "batch", "stream"]:  
    print(metodo, "-> chat_prompt_template_q5:", hasattr(chat_prompt_template_q5, metodo))  
    print(metodo, "-> llm:", hasattr(llm, metodo))
```

```
invoke -> chat_prompt_template_q5: True  
invoke -> llm: True  
batch -> chat_prompt_template_q5: True  
batch -> llm: True  
stream -> chat_prompt_template_q5: True  
stream -> llm: True
```

```
perguntas_q6 = [  
    "O que é uma lista em Python?",  
    "O que é uma tupla em Python?",  
    "O que é um dicionário em Python?",  
]
```

```
batch_responses_q6 = llm.batch(perguntas_q6)
```

```
for resposta in batch_responses_q6:  
    print(resposta.content)  
    print("-" * 40)
```

```

pessoa = {
    "nome": "João",
    "idade": 30,
    "cidade": "São Paulo"
}

# Acessando valores
print(pessoa["nome"]) # Saída: João

# Adicionando um novo par chave-valor
pessoa["profissão"] = "Engenheiro"

# Modificando um valor existente
pessoa["idade"] = 31

# Removendo um par chave-valor
del pessoa["cidade"]

# Exibindo o dicionário atualizado

```

▼ Questão 7 — Construindo uma `chain` com LCEL

Utilizando o operador `|` (LCEL), construa uma `chain` chamada `chain_q7` que:

1. Recebe um dicionário com a chave `"topico"`;
2. Formata o prompt usando `prompt_template_q4` (da Questão 4);
3. Envia o prompt formatado para o `llm`;
4. Usa um `StrOutputParser` para converter a saída (`AIMessage`) em uma `string` simples.

Invoque a `chain` com `{"topico": "programação orientada a objetos"}`, armazene o resultado em `result_q7` e imprima o resultado e o seu tipo (`type(result_q7)`) para confirmar que é uma `string`.

```

from langchain_core.output_parsers import StrOutputParser

chain_q7 = prompt_template_q4 | llm | StrOutputParser()

result_q7 = chain_q7.invoke({"topico": "programação orientada a objetos"})
print(result_q7)
print(type(result_q7))

```

Claro! Vamos imaginar que estamos falando sobre um mundo de objetos, assim como no nosso dia a dia.

O que é Programação Orientada a Objetos (POO)?

A Programação Orientada a Objetos (POO) é uma maneira de organizar e estruturar o código de um programa, usando "objetos".

Vamos entender isso melhor:

1. **Objetos**: Pense em um carro. Um carro tem características (ou propriedades) como cor, modelo e ano. Ele também tem co
2. **Classes**: A classe é como um molde ou uma receita para criar objetos. Usando o exemplo do carro, podemos ter uma clas
3. **Propriedades e Métodos**:
 - **Propriedades**: São as características do objeto. No nosso exemplo, as propriedades do carro seriam a cor, o modelo
 - **Métodos**: São as ações que o objeto pode realizar. Para o carro, os métodos seriam acelerar, frear e buzinar.
4. **Encapsulamento**: Isso significa que as propriedades e métodos de um objeto estão agrupados juntos. Assim, você pode u
5. **Herança**: É uma maneira de criar novas classes a partir de classes existentes. Por exemplo, se tivermos uma classe "V
6. **Polimorfismo**: Isso permite que diferentes objetos respondam de maneiras diferentes à mesma ação. Por exemplo, tanto

Resumindo

A Programação Orientada a Objetos é uma forma de pensar sobre o código como um conjunto de objetos que interagem entre si,

Espero que isso ajude a entender o conceito de POO! Se tiver mais perguntas, fique à vontade para perguntar.

```
<class 'langchain_core.messages.base.TextAccessor'>
```

▼ Questão 8 — `RunnableParallel`

O `RunnableParallel` executa vários `Runnables` ao mesmo tempo, usando a mesma entrada, e retorna um dicionário com o resultado de cada um.

Crie uma chain chamada `parallel_chain_q8`, usando `RunnableParallel`, que receba um dicionário `{"topico": ...}` e produza, em paralelo, **duas explicações diferentes** para o mesmo tópico:

- `"explicacao_simples"`: usando `chain_q7` (a chain criada na Questão 7, que já retorna uma string simples);
- `"explicacao_tecnica"`: uma nova chain que usa um `ChatPromptTemplate` próprio, instruindo o modelo a explicar o tópico de forma técnica e formal (pode usar mensagem de sistema + variável `{topico}`), seguido do mesmo `llm` e de um `StrOutputParser`.

Invoque `parallel_chain_q8` com `{"topico": "ponteiros"}` e imprima as duas explicações retornadas.

```
from langchain_core.runnables import RunnableParallel

chat_prompt_tecnico_q8 = ChatPromptTemplate.from_messages([
    ("system", "Você é um especialista técnico. Explique o tópico solicitado de "
               "forma técnica, formal e precisa, usando terminologia adequada."),
    ("human", "Explique o seguinte tópico: {topico}"),
])

chain_tecnica_q8 = chat_prompt_tecnico_q8 | llm | StrOutputParser()

parallel_chain_q8 = RunnableParallel(
    explicacao_simples=chain_q7,
    explicacao_tecnica=chain_tecnica_q8,
)

result_q8 = parallel_chain_q8.invoke({"topico": "ponteiros"})

print("Explicação simples:\n", result_q8["explicacao_simples"])
print("\nExplicação técnica:\n", result_q8["explicacao_tecnica"])
```

A declaração de um ponteiro é feita utilizando o operador de asterisco (*). Por exemplo, para declarar um ponteiro que :

```
```c
int *p;
```
```

Neste caso, ``p`` é um ponteiro que pode armazenar o endereço de uma variável do tipo ``int``.

Atribuição de Endereços

Para atribuir o endereço de uma variável a um ponteiro, utiliza-se o operador de endereço (``&``). Por exemplo:

```
```c
int x = 10;
int *p = &x;
```
```

Aqui, ``p`` agora contém o endereço de ``x``. É importante notar que o tipo do ponteiro deve corresponder ao tipo da variável

Desreferenciação

A desreferenciação de um ponteiro é feita utilizando o operador de asterisco (*) novamente, mas desta vez na forma de um

```
```c
int y = *p; // y agora é igual a 10
*p = 20; // x agora é igual a 20
```
```

Aritmética de Ponteiros

Ponteiros também suportam operações aritméticas, permitindo a navegação através de arrays e estruturas de dados. A adição

```
```c
```

```
Ponteiros e Alocação Dinâmica
```

Ponteiros são frequentemente utilizados em conjunto com a alocação dinâmica de memória, que é realizada através das funções

```
Conclusão
```

Os ponteiros são uma ferramenta poderosa que proporciona flexibilidade e eficiência na manipulação de dados em C. No entanto,

## ✓ Questão 9 — Structured Output com Pydantic

O método `with_structured_output` do LangChain recebe uma classe Pydantic (`BaseModel`) e retorna uma versão do modelo que devolve diretamente uma instância validada dessa classe, em vez de texto livre.

a) Defina uma classe Pydantic chamada `BugReport` com os seguintes campos:

- `titulo` (`str`): título curto do problema relatado;
- `severidade` (`Literal["baixa", "media", "alta"]`): severidade do bug, inferida a partir do texto;
- `passos_para_reproduzir` (`list[str]`): lista de passos para reproduzir o problema, se mencionados;
- `componente_afetado` (`Optional[str]`): componente ou parte do sistema afetada, se mencionado.

Lembre-se de usar `Field(description=...)` em cada campo, para orientar o modelo sobre o que extrair.

b) Vincule essa classe ao `llm` usando `with_structured_output`, criando `structured_bug_llm`.

c) Invoque `structured_bug_llm` com o texto abaixo e imprima o resultado:

Quando clico no botão "Salvar" da tela de cadastro de clientes, a página trava completamente e preciso recarregar o navegador. Isso acontece toda vez que o campo de telefone está vazio. Esse problema está travando totalmente o módulo de cadastro e impedindo o time de vendas de trabalhar.

```
from typing import Literal, Optional
from pydantic import BaseModel, Field

class BugReport(BaseModel):
 """Structured report of a software bug described in free text."""

 titulo: str = Field(
 description="Título curto e objetivo que resume o problema relatado"
)
 severidade: Literal["baixa", "media", "alta"] = Field(
 description="Severidade do bug, inferida a partir do impacto descrito no texto"
)
 passos_para_reproduzir: list[str] = Field(
 description="Lista ordenada de passos para reproduzir o problema, se mencionados no texto"
)
 componente_afetado: Optional[str] = Field(
 description="Componente, tela ou módulo do sistema afetado, se mencionado"
)
```

```
structured_bug_llm = llm.with_structured_output(BugReport)

texto_bug_q9 = (
 'Quando clico no botão "Salvar" da tela de cadastro de clientes, a página trava '
 'completamente e preciso recarregar o navegador. Isso acontece toda vez que o '
 'campo de telefone está vazio. Esse problema está travando totalmente o módulo '
 'de cadastro e impedindo o time de vendas de trabalhar.'
)

bug_report_q9 = structured_bug_llm.invoke(texto_bug_q9)
print(bug_report_q9)

azio' severidade='alta' passos_para_reproduzir=['Acessar a tela de cadastro de clientes.', 'Deixar o campo de telefone vazio']
```

## ✓ Questão 10 — Mini pipeline: da mensagem à saída estruturada

Nesta última questão, você vai combinar vários conceitos vistos nas questões anteriores em um único fluxo.

Crie uma classe Pydantic chamada `Feedback` com os campos:

- `sentimento` (`Literal["positivo", "negativo", "neutro"]`);
- `pontos_positivos` (`list[str]`);
- `pontos_negativos` (`list[str]`);
- `resumo` (`str`, com no máximo uma frase).

Em seguida:

1. Crie um `ChatPromptTemplate` chamado `chat_prompt_q10`, com uma mensagem de sistema instruindo o modelo a **analisar feedbacks de clientes** e uma mensagem humana com uma variável `{feedback_cliente}`.
2. Crie um modelo estruturado `structured_feedback_llm`, vinculando a classe `Feedback` ao `llm` com `with_structured_output`.
3. Construa uma chain chamada `chain_q10`, usando o operador `|`, que encadeia `chat_prompt_q10` com `structured_feedback_llm` (sem `StrOutputParser`, já que a saída aqui deve continuar sendo um objeto `Feedback`).
4. Invoque `chain_q10` com o feedback abaixo e imprima o resultado, além de acessar e imprimir individualmente os campos `sentimento` e `resumo`.

O atendimento foi rápido e a equipe muito educada, mas o produto chegou com a caixa amassada e um dos itens veio riscado. No geral, o suporte pós-venda resolveu bem o problema quando entrei em contato.

```
class Feedback(BaseModel):
 """Structured analysis of a customer feedback message."""

 sentimento: Literal["positivo", "negativo", "neutro"] = Field(
 description="Sentimento geral do feedback, considerando o balanço entre elogios e reclamações"
)
 pontos_positivos: list[str] = Field(
 description="Lista de aspectos positivos mencionados no feedback"
)
 pontos_negativos: list[str] = Field(
 description="Lista de aspectos negativos ou problemas mencionados no feedback"
)
 resumo: str = Field(
 description="Resumo do feedback em no máximo uma frase"
)
```

```
chat_prompt_q10 = ChatPromptTemplate.from_messages([
 ("system", "Você é um analista de feedbacks de clientes. Analise o feedback "
 "fornecido, identificando o sentimento geral, os pontos positivos e "
 "negativos, e produzindo um resumo conciso."),
 ("human", "Analise o seguinte feedback de cliente:\n\n{feedback_cliente}"),
])

structured_feedback_llm = llm.with_structured_output(Feedback)

chain_q10 = chat_prompt_q10 | structured_feedback_llm

feedback_cliente_q10 = (
 "O atendimento foi rápido e a equipe muito educada, mas o produto chegou com a "
 "caixa amassada e um dos itens veio riscado. No geral, o suporte pós-venda "
 "resolveu bem o problema quando entrei em contato."
)

result_q10 = chain_q10.invoke({"feedback_cliente": feedback_cliente_q10})

print(result_q10)
print("Sentimento:", result_q10.sentimento)
print("Resumo:", result_q10.resumo)
```

```
sentimento='positivo' pontos_positivos=['Atendimento rápido', 'Equipe educada', 'Bom suporte pós-venda'] pontos_negativos=[
Sentimento: positivo
Resumo: O atendimento foi eficiente e educado, mas o produto apresentou danos na entrega.
```

